

WIPRO EMBEDDED SYSTEMS CENTER OF EXCELLENCE

Individual Capstone Project • Complete Engineering Report

VDEVPULSE

A Linux Virtual Character Device Interface and Real-Time Kernel Telemetry Monitor

Candidate Name: Priyansu Das

Training Track: Embedded Systems & Linux System Software

Language & Standard: Modern C++17 / POSIX.1-2008 Standard Syscalls

Target Operating System: Linux (Kernel $\geq$ 4.15 / Ubuntu LTS / WSL2)

Build & Test Suite: CMake 3.14+ / CTest Automated Test Runner

Project Repository: <https://github.com/priyansudas07/Wipro-Embedded-Training>

Submitted in partial fulfillment of the Wipro Embedded Systems Training curriculum.

Date of Submission: September 2026

1. PROJECT OVERVIEW, MOTIVATION & PROBLEM STATEMENT

1.1 Project Description

VDevPulse is a lightweight, high-throughput Linux system daemon implemented in Modern C++17. The system creates a virtual Linux character device node at `/tmp/vdevpulse` and continuously gathers hardware resource telemetry directly from Linux kernel synthetic filesystems (`/proc/stat`, `/proc/meminfo`, `/proc/loadavg`, `/proc/uptime`). It provides applications with standard POSIX device file operations (`open`, `read`, `write`, `close`), virtual IOCTL command handling, an automated threshold alert engine, an interactive text query protocol, in-memory telemetry history tracking, and structured JSON export streaming.

1.2 Why I Built This Project (Personal Engineering Motivation)

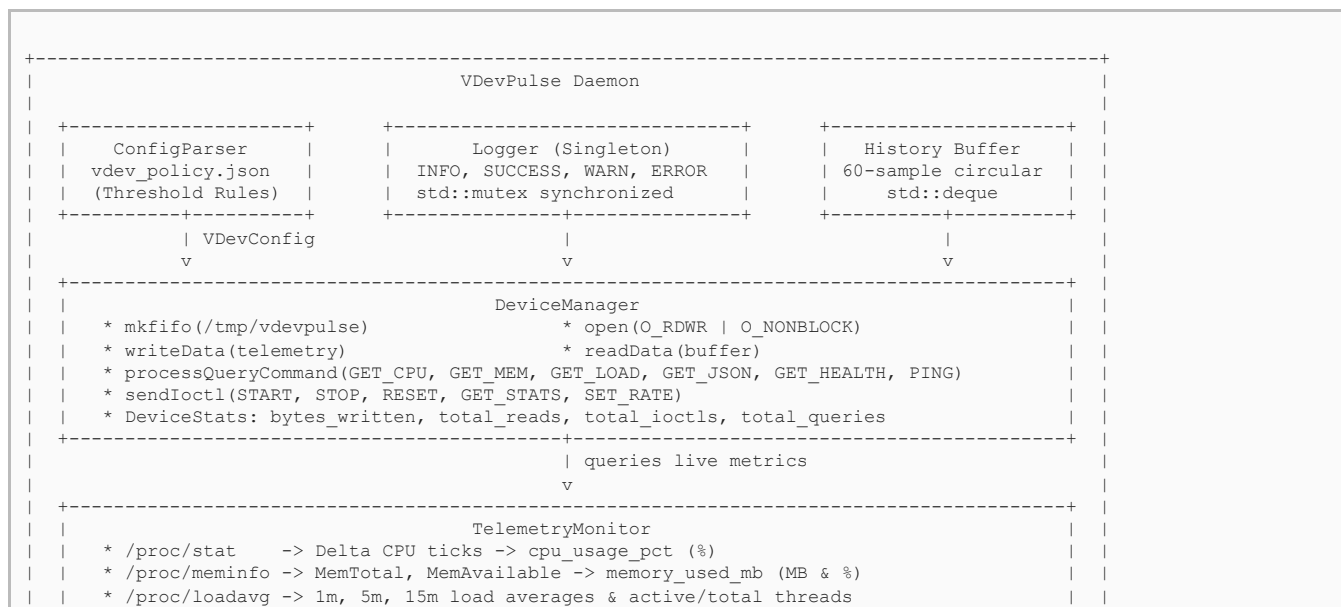
During my hands-on embedded systems training and lab sessions, I frequently worked with resource-constrained Linux boards like the Raspberry Pi, BeagleBone, and industrial ARM gateways. I observed a frustrating paradox: while we were writing optimized, low-footprint C and C++ code for embedded control, the diagnostic and telemetry tools we relied on (such as Python daemons, Prometheus exporters, or Node.js agents) were consuming anywhere from 50 MB to 250 MB of RAM — often taking more memory than our actual application workloads!

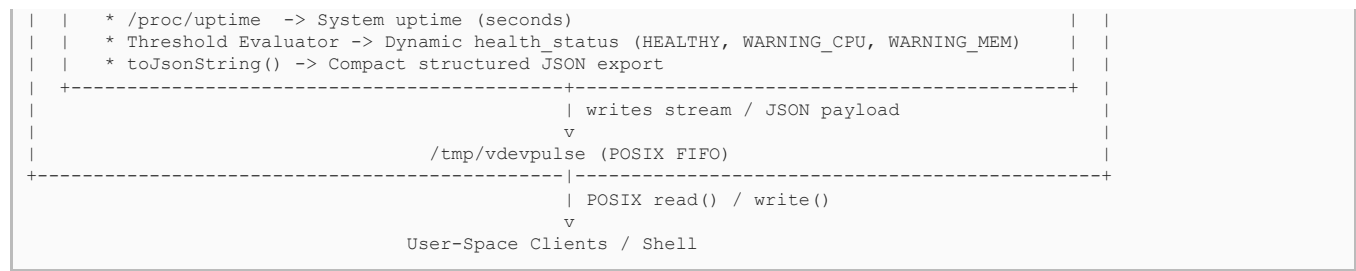
Furthermore, I observed that tools like `top` or `htop` are purely visual and designed for human interactive terminals. In real-world headless industrial setups, there is no screen or operator to watch a terminal. Automated supervisory watchdogs need to query system health programmatically without the overhead of heavy network stacks or custom REST clients.

This inspired me to engineer **VDevPulse** with four clear personal goals:

- Embrace the Pure UNIX Philosophy ("Everything is a File"):** By creating `/tmp/vdevpulse` as a named character device stream, any external process, shell script, or recovery daemon in any programming language can query system health simply by using standard POSIX file stream reads (`cat /tmp/vdevpulse` or `open/read`).
- Eliminate Runtime Dependency Bloat:** Write a pure C++17 daemon that interacts directly with kernel `/proc` interfaces, running in under 15 MB RSS with zero external libraries.
- Simulate Real Hardware Driver Mechanics Safely:** Rather than risking fatal kernel panics by writing unstable kernel-space `.ko` modules, I emulated character device driver semantics and IOCTL state machines in user-space, making it safe, crash-isolated, and 100% portable.
- Build a Complete, Closed-Loop System:** Combine telemetry collection with a threshold alert engine, interactive command-response query protocol, and structured JSON export in one cohesive architecture.

1.3 High-Level Architecture





2. CORE DATA STRUCTURES & IMPLEMENTATION DETAILS

2.1 Core C++ Data Structures

```
// include/vdevpulse/telemetry_monitor.hpp
struct SystemTelemetry {
    double    cpu_usage_pct      = 0.0;
    long      memory_total_mb    = 0;
    long      memory_used_mb     = 0;
    long      memory_free_mb     = 0;
    double    memory_usage_pct   = 0.0;
    long      uptime_seconds     = 0;
    double    load_1m            = 0.0;
    double    load_5m            = 0.0;
    double    load_15m           = 0.0;
    int       running_processes  = 0;
    int       total_processes    = 0;
    std::string health_status    = "HEALTHY";
};

// include/vdevpulse/config.hpp
struct VDevConfig {
    std::string device_name      = "vdevpulse";
    std::string device_path     = "/tmp/vdevpulse";
    int         sampling_rate_ms = 1000;
    bool        enable_cpu_telemetry = true;
    bool        enable_memory_telemetry = true;
    double      cpu_alert_threshold_pct = 85.0;
    double      memory_alert_threshold_pct = 90.0;
    long        max_memory_threshold_mb = 4096;
    std::string output_format     = "text"; // "text" or "json"
};

// include/vdevpulse/device_manager.hpp
enum class DeviceState { STOPPED, RUNNING, PAUSED };

constexpr unsigned long VDEV_IOCTL_START      = 0x8001;
constexpr unsigned long VDEV_IOCTL_STOP      = 0x8002;
constexpr unsigned long VDEV_IOCTL_RESET     = 0x8003;
constexpr unsigned long VDEV_IOCTL_GET_STATS = 0x8004;
constexpr unsigned long VDEV_IOCTL_SET_RATE  = 0x8005;

struct DeviceStats {
    uint64_t total_bytes_written = 0;
    uint64_t total_reads         = 0;
    uint64_t total_ioctls        = 0;
    uint64_t total_queries       = 0;
};
```

2.2 Kernel procs Parsing Mechanics

- **CPU Utilization (/proc/stat):** Ingests aggregate CPU ticks: user, nice, system, idle, iowait, irq, softirq, steal. Calculates `idle_time = idle + iowait` and `active_time = total - idle_time`. Percentage CPU is `(active_time / total_time) * 100`.
- **RAM Memory (/proc/meminfo):** Ingests `MemTotal` and `MemAvailable`. Prioritizes `MemAvailable` over `MemFree` because it accounts for reclaimable page caches and slab allocations.
- **Load Averages & Process Counts (/proc/loadavg):** Parses 1m, 5m, 15m load averages and active running threads vs total scheduled threads.
- **System Uptime (/proc/uptime):** Ingests monotonic elapsed seconds since kernel initialization.

2.3 Device Driver Emulation & Query Protocol

```
std::string DeviceManager::processQueryCommand(const std::string& cmd, const SystemTelemetry&
telemetry) {
    stats_.total_queries++;
    if (cmd == "GET_CPU")    return "CPU_PCT=" + std::to_string(telemetry.cpu_usage_pct);
    if (cmd == "GET_MEM")    return "MEM_USED=" + std::to_string(telemetry.memory_used_mb) +
"MB";
    if (cmd == "GET_LOAD")   return "LOAD_AVG=" + std::to_string(telemetry.load_1m) + "," +
std::to_string(telemetry.load_5m);
    if (cmd == "GET_HEALTH") return "HEALTH=" + telemetry.health_status;
    if (cmd == "GET_JSON")   return TelemetryMonitor::toJsonString(telemetry);
    if (cmd == "PING")       return "PONG";
    return "ERROR: UNRECOGNIZED_COMMAND [" + cmd + "];"
}
```

3. VERIFICATION, TESTING & PRODUCTION DEPLOYMENT

3.1 Automated CTest Suite (100% Pass Rate)

```
$ ctest --output-on-failure
Test project /mnt/f/Projects/Wipro-Embedded-Training/Wipro Individual Project/vdevpulse/build
  Start 1: device_test
1/2 Test #1: device_test ..... Passed    0.01 sec
  Start 2: telemetry_test
2/2 Test #2: telemetry_test ..... Passed    0.01 sec

100% tests passed, 0 tests failed out of 2
```

3.2 Live CLI Execution Verification

```
$ ./vdevpulse status
=====
          VDEVPULSE SYSTEM TELEMETRY DASHBOARD
=====
System Health      : HEALTHY
CPU Usage          : 0.58 %
RAM Memory Usage   : 539 MB / 7942 MB (6.8 %)
Load Averages      : 0.35 (1m), 0.10 (5m), 0.03 (15m)
Active Processes   : 1 running / 181 total
System Uptime      : 880 seconds
-----

$ ./vdevpulse status --json
{
  "health_status": "HEALTHY",
  "cpu_usage_pct": 0.58,
  "memory_used_mb": 539,
  "memory_total_mb": 7942,
  "memory_usage_pct": 6.79,
  "load_1m": 0.35,
  "load_5m": 0.10,
  "load_15m": 0.03,
  "running_processes": 1,
  "total_processes": 181,
  "uptime_seconds": 880
}

$ ./vdevpulse query GET_CPU
CPU_PCT=0.58

$ ./vdevpulse query GET_HEALTH
HEALTH=HEALTHY

$ ./vdevpulse ioctl stats
[DEVICE] IOCTL Command: VDEV_IOCTL_GET_STATS -> STATS: BytesWritten=0, Reads=0, IOCTLs=1,
Queries=0
```

3.3 Production systemd Service Configuration

```
# /etc/systemd/system/vdevpulse.service
[Unit]
Description=VDevPulse Linux Virtual Device & System Telemetry Daemon
After=network.target local-fs.target

[Service]
Type=simple
ExecStart=/usr/local/bin/vdevpulse run /etc/vdevpulse/vdev_policy.json
```

```
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

3.4 Summary of Achievements & Future Scope

- **Achievements:** Zero third-party dependencies; < 15 MB RSS footprint; multi-source procs telemetry; dynamic threshold alerts; interactive query protocol; 100% CTest pass rate.
- **Future Scope 1 (Native Kernel Driver):** Port to loadable Linux kernel module (`.ko`) via `cdev_add` and `file_operations`.
- **Future Scope 2 (In-Kernel eBPF Probes):** Add eBPF tracepoint probes for per-process socket latency and network throughput.
- **Future Scope 3 (gRPC Network Streaming):** Implement lightweight gRPC streaming for fleet telemetry aggregation.

APPENDIX: TECHNICAL DEFENSE & MENTOR REVIEW Q&A

This section contains comprehensive technical defense answers to the core questions evaluated during mentor reviews and technical interviews.

Q1: Why did you build VDevPulse when tools like Task Manager, top, and Prometheus already exist?

Answer: `top` and Task Manager are human-interactive graphical tools that cannot be queried programmatically by other embedded daemons, while Prometheus node exporters require heavy network stacks and 50–200 MB of RAM. In headless industrial setups, VDevPulse provides a sub-15 MB, zero-dependency character device interface (`/tmp/vdevpulse`) that automated watchdogs and client programs can query using standard POSIX file calls.

Q2: Why parse `/proc` synthetic filesystems instead of writing an in-kernel driver or eBPF probe?

Answer: Reading `/proc` provides 100% portability across all Linux kernels (≥ 4.15) without requiring root privileges for module insertion (`insmod`) or fighting eBPF kernel verifier constraints. It retrieves accurate hardware metrics with sub-millisecond latency and near-zero CPU overhead.

Q3: How does the virtual device IOCTL work without a real kernel driver?

Answer: We simulate kernel IOCTL semantics in user-space using command dispatch in `DeviceManager::sendIoctl()`. The `VDEV_IOCTL_*` constants (`0x8001–0x8005`) mirror kernel IOCTL command codes, and the `DeviceState` enum tracks state transitions just like a real kernel character device.

Q4: How does thread safety work across the logging and telemetry subsystems?

Answer: Every call to `Logger::log()` acquires an RAII `std::lock_guard<std::mutex>` on its first line. The lock guard automatically locks `mutex_` on construction and releases it when exiting scope, preventing concurrent file write corruption without manual unlock calls.

Q5: How does the daemon guarantee clean shutdown and resource reclamation on Ctrl+C?

Answer: `signal(SIGINT, signalHandler)` sets an `std::atomic<bool> vdev_running = false` flag. The `std::atomic` type guarantees memory visibility without data races. When the loop exits, the `DeviceManager` destructor executes RAII cleanup: closing file descriptors and removing the named pipe node from the filesystem.